# Integration Manual

for S32K3 CRC Driver

Document Number: IM34CRCASR4.4 Rev0000R 2.0.0 Rev. 1.0

**1 Revision History** **2**

**2 Introduction** **3**

2.1 Supported Derivatives . . . 3
2.2 Overview . . . 4
2.3 About This Manual . . . 4
2.4 Acronyms and Definitions . . . 5
2.5 Reference List . . . 5

**3 Building the driver** **6**

3.1 Build Options . . . 6
3.1.1 GCC Compiler/Assembler/Linker Options . . . 6
3.1.2 GHS Compiler/Assembler/Linker Options . . . 9
3.1.3 IAR Compiler/Assembler/Linker Options . . . 11
3.2 Files required for compilation . . . 13
3.2.1 Crc Driver Files . . . 13
3.2.2 As a deviation from standard: . . . 14
3.2.3 Files from Base common folder . . . 14
3.2.4 Files from Det folder: . . . 14
3.2.5 RTE Files: . . . 14
3.3 Setting up the plugins . . . 14

**4 Function calls to module** **16**

4.1 Function Calls during Start-up . . . 16
4.2 Function Calls during Shutdown . . . 16
4.3 Function Calls during Wake-up . . . 16

**5 Module requirements** **17**

5.1 Exclusive areas to be defined in BSW scheduler . . . 17
5.2 Exclusive areas not available on this platform . . . 17
5.3 Peripheral Hardware Requirements . . . 17
5.4 ISR to configure within AutosarOS - dependencies . . . 17
5.5 ISR Macro . . . 18
5.5.1 Without an Operating System . . . 18
5.5.2 With an Operating System . . . 18
5.6 Other AUTOSAR modules - dependencies . . . 18
5.7 Data Cache Restrictions . . . 19
5.8 User Mode support . . . 19
5.8.1 User Mode configuration in the module . . . 19
5.8.2 User Mode configuration in AutosarOS . . . 19
5.9 Multicore support . . . 20

**6 Main API Requirements** **21**
6.1 Main function calls within BSW scheduler . . . 21
6.2 API Requirements . . . 21
6.3 Calls to Notification Functions, Callbacks, Callouts . . . 21

**7 Memory allocation** **22**
7.1 Sections to be defined in CRC_MemMap.h . . . 22
7.2 Linker command file . . . 23

**8 Integration Steps** **24**

**9 External assumptions for driver** **25**

# Chapter 1

# Revision History

| Revision | Date | Author | Description |
|---|---|---|---|
| 1.0 | 31.03.2022 | NXP RTD Team | Prepared for release RTD S32K3 2.0.0 |

# Chapter 2

# Introduction


- Supported Derivatives
- Overview
- About This Manual
- Acronyms and Definitions
- Reference List


This integration manual describes the integration requirements for CRC Driver for S32K3XX microcontrollers.

## 2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k344_mapbga257
- s32k344_mqfp172
- s32k324_mapbga257
- s32k324_mqfp172
- s32k314_mapbga257
- s32k314_mqfp172
- s32k312_mqfp172
- s32k312_mqfp100
- s32k322_mqfp172
- s32k322_mqfp100
- s32k342_mqfp172
- s32k342_mqfp100
- s32k341_mqfp172
- s32k341_mqfp100

All of the above microcontroller devices are collectively named as S32K3.

## 2.2 Overview

**AUTOSAR (AUTomotive Open System ARchitecture)** is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

## 2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

## 2.4 Acronyms and Definitions

| Term | Definition |
|---|---|
| API | Application Programming Interface |
| ASM | Assembler |
| BSMI | Basic Software Make file Interface |
| CAN | Controller Area Network |
| C/CPP | C and C++ Source Code |
| CS | Chip Select |
| CTU | Cross Trigger Unit |
| DEM | Diagnostic Event Manager |
| DET | Development Error Tracer |
| DMA | Direct Memory Access |
| ECU | Electronic Control Unit |
| FIFO | First In First Out |
| LSB | Least Signifigant Bit |
| MCU | Micro Controller Unit |
| MIDE | Multi Integrated Development Environment |
| MSB | Most Significant Bit |
| N/A | Not Applicable |
| RAM | Random Access Memory |
| SIU | Systems Integration Unit |
| SWS | Software Specification |
| VLE | Variable Length Encoding |
| XML | Extensible Markup Language |

## 2.5 Reference List

| # | Title | Version |
|---|---|---|
| 1 | Specification of CRC Driver | AUTOSAR Release 4.4.0 |
| 2 | Hardware Safety Manual | S32K3xx Safety Manual, Rev. 1, 09/2021 |
| 3 | Reference Manual | S32K3xx Reference Manual, Rev. 3, 10/2021 |
| 4 | Datasheet | S32K3xx Data Sheet, Rev. 3 — 10/2021 |
| 5 | Errata | S32K312: Mask Set Errata for Mask 0P09C, Rev. 10 Jan 2022<br>S32K342: Mask Set Errata for Mask 0P97C, Rev. 13 , Rev. 13 Dec 2021<br>S32K344: Mask Set Errata for Mask 0P55A/1P55A, Rev. 30 Sep 2021 |

# Chapter 3

# Building the driver

- Build Options
- Files required for compilation
- Setting up the plugins

This section describes the source files and various compilers, linker options used for building the driver.
It also explains the EB Tresos Studio plugin setup procedure.

## 3.1 Build Options

- GCC Compiler/Assembler/Linker Options
- GHS Compiler/Assembler/Linker Options
- IAR Compiler/Assembler/Linker Options

The RTD driver files are compiled using:

- NXP GCC 10.2.0 20200723 (Build 1728 Revision g5963bc8)
- IAR ANSI C/C++ Compiler V8.50.10.290/W32 for ARM Functional Safety
- Green Hills Multi 7.1.6d / Compiler 2021.1.4

The compiler, assembler, and linker flags used for building the driver are explained below.
The TS_T40D34M20I0R0 part of the plugin name is composed as follows:

- T = Target_Id (e.g. T40 identifies Cortex-M architecture)
- D = Derivative_Id (e.g. D34 identifies S32K3 platform)
- M = SW_Version_Major and SW_Version_Minor
- I = SW_Version_Patch
- R = Reserved

### 3.1.1 GCC Compiler/Assembler/Linker Options

#### 3.1.1.1 GCC Compiler Options

| Compiler Option | Description |
|---|---|
| -mcpu=cortex-m7 | Targeted ARM processor for which GCC should tune the performance of the code |
| -mthumb | Generates code that executes in Thumb state |
| -mlittle-endian | Generate code for a processor running in little-endian mode |
| -mfpu=fpv5-sp-d16 | Specifies the floating-point hardware available on the target |
| -mfloat-abi=hard | Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions |
| -std=c99 | Specifies the ISO C99 base standard |
| -Os | Optimize for size. Enables all -O2 optimizations except those that often increase code size |
| -ggdb3 | Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program |
| -Wall | Enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros |
| -Wextra | This enables some extra warning flags that are not enabled by -Wall |
| -pedantic | Issue all the warnings demanded by strict ISO C. Reject all programs that use forbidden extensions. Follows the version of the ISO C standard specified by the aforementioend -std option |
| -Wstrict-prototypes | Warn if a function is declared or defined without specifying the argument types |
| -Wundef | Warn if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero |
| -Wunused | Warn whenever a function, variable, label, value, macro is unused |
| -Werror=implicit-function-declaration | Make the specified warning into an error. This option throws an error when a function is used before being declared |
| -Wsign-compare | Warn when a comparison between signed and unsigned values could produce an incorrect result when the signed value is converted to unsigned. |
| -Wdouble-promotion | Give a warning when a value of type float is implicitly promoted to double |
| -fno-short-enums | Specifies that the size of an enumeration type is at least 32 bits regardless of the size of the enumerator values. |
| -funsigned-char | Let the type char be unsigned by default, when the declaration does not use either signed or unsigned |
| -funsigned-bitfields | Let a bit-field be unsigned by default, when the declaration does not use either signed or unsigned |
| -fomit-frame-pointer | Omit the frame pointer in functions that dont need one. This avoids the instructions to save, set up and restore the frame pointer; on many targets it also makes an extra register available. |

| Compiler Option | Description |
|---|---|
| -fno-common | Makes the compiler place uninitialized global variables in the BSS section of the object file. This inhibits the merging of tentative definitions by the linker so you get a multiple-definition error if the same variable is accidentally defined in more than one compilation unit |
| -fstack-usage | Makes the compiler output stack usage information for the program, on a per-function basis |
| -fdump-ipa-all | Enables all inter-procedural analysis dumps |
| -c | Stop after assembly and produce an object file for each source file |
| -DS32K3XX | Predefine S32K3XX as a macro, with definition 1 |
| -DS32K344 | Predefine S32K344 as a macro, with definition 1 |
| -DGCC | Predefine GCC as a macro, with definition 1 |
| -DUSE_SW_VECTOR_MODE | Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode |
| -DD_CACHE_ENABLE | Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initalization in source file system.↩ c under the Platform driver |
| -DI_CACHE_ENABLE | Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initalization in source file system.c under the Platform driver |
| -DENABLE_FPU | Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initalization in source file system.c under the Platform driver |
| -DMCAL_ENABLE_USER_MODE_SUPPORT | Predefine MCAL_ENABLE_USER_MODE_SUPPO↩ RT as a macro, with definition 1. Allows drivers to be configured in user mode. |

#### 3.1.1.2 GCC Assembler Options

| Assembler Option | Description |
|---|---|
| -Xassembler-with-cpp | Specifies the language for the following input files (rather than letting the compiler choose a default based on the file name suffix) |
| -mcpu=cortexm7 | Targeted ARM processor for which GCC should tune the performance of the code |
| -mthumb | Generates code that executes in Thumb state |
| -c | Stop after assembly and produce an object file for each source file |

#### 3.1.1.3 GCC Linker Options

| Linker Option | Description |
|---|---|
| -Wl,-Map,filename | Produces a map file |
| -T linkerfile | Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it) |
| –entry=Reset_Handler | Specifies that the program entry point is Reset_Handler |

| Linker Option | Description |
|---|---|
| -nostartfiles | Do not use the standard system startup files when linking |
| -mcpu=cortexm7 | Targeted ARM processor for which GCC should tune the performance of the code |
| -mthumb | Generates code that executes in Thumb state |
| -mfpu=fpv5-sp-d16 | Specifies the floating-point hardware available on the target |
| -mfloat-abi=hard | Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions |
| -mlittle-endian | Generate code for a processor running in little-endian mode |
| -ggdb3 | Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program |
| -lc | Link with the C library |
| -lm | Link with the Math library |
| -lgcc | Link with the GCC library |

### 3.1.2 GHS Compiler/Assembler/Linker Options

#### 3.1.2.1 GHS Compiler Options

| Compiler Option | Description |
|---|---|
| -cpu=cortexm7 | Selects target processor: Arm Cortex M7 |
| -thumb | Selects generating code that executes in Thumb state |
| -fpu=vfpv5_d16 | Specifies hardware floating-point using the v5 version of the VFP instruction set, with 16 double-precision floating-point registers |
| -fsingle | Use hardware single-precision, software double-precision FP instructions |
| -C99 | Use (strict ISO) C99 standard (without extensions) |
| –ghstd=last | Use the most recent version of Green Hills Standard mode (which enables warnings and errors that enforce a stricter coding standard than regular C and C++) |
| -Osize | Optimize for size |
| –gnu_asm | Enables GNU extended asm syntax support |
| -dual_debug | Generate DWARF 2.0 debug information |
| -G | Generate debug information |
| -keeptempfiles | Prevents the deletion of temporary files after they are used. If an assembly language file is created by the compiler, this option will place it in the current directory instead of the temporary directory |
| -Wimplicit-int | Produce warnings if functions are assumed to return int |
| -Wshadow | Produce warnings if variables are shadowed |
| -Wtrigraphs | Produce warnings if trigraphs are detected |
| -Wundef | Produce a warning if undefined identifiers are used in #if preprocessor statements |
| –unsigned_chars | Let the type char be unsigned, like unsigned char |

| Compiler Option | Description |
|---|---|
| –unsigned_fields | Bitfelds declared with an integer type are unsigned |
| –no_commons | Allocates uninitialized global variables to a section and initializes them to zero at program startup |
| –no_exceptions | Disables C++ support for exception handling |
| –no_slash_comment | C++ style // comments are not accepted andgenerate errors |
| –prototype_errors | Controls the treatment of functions referenced or called when no prototype has been provided |
| –incorrect_pragma_warnings | Controls the treatment of valid #pragma directives that use the wrong syntax |
| -c | Stop after assembly and produce an object file for each source file |
| -DS32K3XX | Predefine S32K3XX as a macro, with definition 1 |
| -DS32K344 | Predefine S32K344 as a macro, with definition 1 |
| -DGHS | Predefine GHS as a macro, with definition 1 |
| -DUSE_SW_VECTOR_MODE | Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode |
| -DD_CACHE_ENABLE | Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initalization in source file system.↩ c under the Platform driver |
| -DI_CACHE_ENABLE | Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initalization in source file system.c under the Platform driver |
| -DENABLE_FPU | Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initalization in source file system.c under the Platform driver |
| -DMCAL_ENABLE_USER_MODE_SUPPORT | Predefine MCAL_ENABLE_USER_MODE_SUPPO↩ RT as a macro, with definition 1. Allows drivers to be configured in user mode |

#### 3.1.2.2 GHS Assembler Options

| Assembler Option | Description |
|---|---|
| -cpu=cortexm7 | Selects target processor: Arm Cortex M7 |
| -preprocess_assembly_files | Controls whether assembly files with standard extensions such as .s and .asm are preprocessed |
| -list | Creates a listing by using the name and directory of the object file with the .lst extension |
| -c | Stop after assembly and produce an object file for each source file |

#### 3.1.2.3 GHS Linker Options

| Linker Option | Description |
|---|---|
| -e Reset_Handler | Make the symbol Reset_Handler be treated as a root symbol and the start label of the application |

| Linker Option | Description |
|---|---|
| -T linker_script_file.ld | Use linker_script_file.ld as the linker script. This script replaces the default linker script (rather than adding to it) |
| -map | Produce a map file |
| -keepmap | Controls the retention of the map file in the event of a link error |
| -Mn | Generates a listing of symbols sorted alphabetically/numerically by address |
| -delete | Instructs the linker to remove functions that are not referenced in the final executable. The linker iterates to find functions that do not have relocations pointing to them and eliminates them |
| -ignore_debug_references | Ignores relocations from DWARF debug sections when using -delete. DWA↩ RF debug information will contain references to deleted functions that may break some third-party debuggers |
| -Llibrary_path | Points to library_path (the libraries location) for thumb2 to be used for linking |
| -larch | Link architecture specific library |
| -lstartup | Link run-time environment startup routines. The source code for themodules in this library is provided in the src/libstartup directory |
| -lind_sd | Link language-independent library, containing support routines for features such as software floating point, run-time error checking, C99 complex numbers, and some general purpose routines of the ANSI C library |
| -v | Prints verbose information about the activities of the linker, including the libraries it searches to resolve undefined symbols |
| -keep=C40_Ip_AccessCode | Avoid linker remove function C40_Ip_AccessCode from Fls module because it is not referenced explicitly |
| -nostartfiles | Controls the start files to be linked into the executable |

### 3.1.3 IAR Compiler/Assembler/Linker Options

#### 3.1.3.1 IAR Compiler Options

| Compiler Option | Description |
|---|---|
| –cpu=Cortex-M7 | Targeted ARM processor for which IAR should tune the performance of the code |
| –cpu_mode=thumb | Generates code that executes in Thumb state |
| –endian=little | Generate code for a processor running in little-endian mode |
| –fpu=FPv5-SP | Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant. |
| -e | Enables all IAR C language extensions |
| -Ohz | Optimize for size. the compiler will emit AEABI attributes indicating the requested optimization goal. This information can be used by the linker to select smaller or faster variants of DLIB library functions |
| –debug | Makes the compiler include debugging information in the object modules. Including debug information will make the object files larger |

| Compiler Option | Description |
|---|---|
| –no_clustering | Disables static clustering optimizations. Static and global variables defined within the same module will not be arranged so that variables that are accessed in the same function are close to each other |
| –no_mem_idioms | Makes the compiler not optimize certain memory access patterns |
| –do_explicit_zero_opt_in_named_sections | Disable the exception for variables in user-named sections, and thus treat explicit initializations to zero as zero initializations, not copy initializations |
| –require_prototypes | Force the compiler to verify that all functions have proper prototypes. Generates an error otherwise |
| –no_wrap_diagnostics | Does not wrap long lines in diagnostic messages |
| –diag_suppress=Pa050 | Suppresses diagnostic message Pa050 |
| -DS32K3XX | Predefine S32K3XX as a macro, with definition 1 |
| -DS32K344 | Predefine S32K344 as a macro, with definition 1 |
| -DIAR | Predefine IAR as a macro, with definition 1 |
| -DUSE_SW_VECTOR_MODE | Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode. |
| -DD_CACHE_ENABLE | Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initalization in source file system.↩ c under the Platform driver |
| -DI_CACHE_ENABLE | Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initalization in source file system.c under the Platform driver |
| -DENABLE_FPU | Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initalization in source file system.c under the Platform driver |
| -DMCAL_ENABLE_USER_MODE_SUPPORT | Predefine MCAL_ENABLE_USER_MODE_SUPPO↩ RT as a macro, with definition 1. Allows drivers to be configured in user mode. |

#### 3.1.3.2 IAR Assembler Options

| Assembler Option | Description |
|---|---|
| –cpu Cortex-M7 | Targeted ARM processor for which IAR should generate the instruction set |
| –cpu_mode thumb | Selects the thumb mode for the assembler directive CODE |
| -g | Disables the automatic search for system include files |
| -r | Generates debug information |

#### 3.1.3.3 IAR Linker Options

| Linker Option | Description |
|---|---|
| –map filename | Produces a map file |
| –config linkerfile | Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it) |

| Linker Option | Description |
|---|---|
| –cpu=Cortex-M7 | Selects the ARM processor variant to link the application for |
| –fpu=FPv5-SP | Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant. |
| –entry _start | Treats _start as a root symbol and start label |
| –enable_stack_usage | Enables stack usage analysis. If a linker map file is produced, a stack usage chapter is included in the map file |
| –skip_dynamic_initialization | Dynamic initialization (typically initialization of C++ objects with static storage duration) will not be performed automatically during application startup |
| –no_wrap_diagnostics | Does not wrap long lines in diagnostic messages |

## 3.2 Files required for compilation

This section describes the include files required to compile, assemble (if assembler code) and link the CRC driver for S32K3 microcontrollers. To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJOR_VERSION and AR_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

This section describes the include files required to compile, assemble (if assembler code) and link the CRC driver for S32K3 microcontrollers.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJO↩ R_VERSION and AR_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

### 3.2.1 Crc Driver Files

- Source files:
  - ..\Crc_TS_T40D34M20I0R0\src\CDD_Crc.c
  - ..\Crc_TS_T40D34M20I0R0\src\Crc_Ip.c
  - ..\Crc_TS_T40D34M20I0R0\src\Crc_Ip_Hardware.c
  - ..\Crc_TS_T40D34M20I0R0\src\Crc_Ip_Lookup_Tables.c
  - ..\Crc_TS_T40D34M20I0R0\src\Crc_Ip_Software.c
- Include file:
  - ..\Crc_TS_T40D34M20I0R0\include\CDD_Crc.h
  - ..\Crc_TS_T40D34M20I0R0\include\Crc_Ip_Devassert.h
  - ..\Crc_TS_T40D34M20I0R0\include\Crc_Ip_Hardware.h
  - ..\Crc_TS_T40D34M20I0R0\include\Crc_Ip_Hw_Access.h
  - ..\Crc_TS_T40D34M20I0R0\include\Crc_Ip_Lib.h
  - ..\Crc_TS_T40D34M20I0R0\include\Crc_Ip_Lookup_Tables.h
  - ..\Crc_TS_T40D34M20I0R0\include\Crc_Ip_Software.h
  - ..\Crc_TS_T40D34M20I0R0\include\Crc_Ip_State.h
  - ..\Crc_TS_T40D34M20I0R0\include\Crc_Ip_Types.h
  - ..\Crc_TS_T40D34M20I0R0\include\Crc_Ip.h
  - ..\Crc_TS_T40D34M20I0R0\include\Crc_Ipw.h
  - ..\Crc_TS_T40D34M20I0R0\include\Crc_Types.h
  - ..\Crc_TS_T40D34M20I0R0\include\Crc.h

### 3.2.2 As a deviation from standard:

- Crc_IP_Cfg.c - This file will contain the definition for all configuration structures containing only variables that are not variant aware, configured and generated only once. This file alone does not contain the whole structure needed by Port_Init function to configure the driver. Based on the number of variants configured in the EcuC, there can be more than one configuration structure for one module even for PreCompile variant.
- Crc_IP_Cfg.h - This file will contain the definition for all configuration structures containing only variables that are not variant aware, configured and generated only once. This file alone does not contain the whole structure needed by Port_Init function to configure the driver. Based on the number of variants configured in the EcuC, there can be more than one configuration structure for one module even for PreCompile variant.

### 3.2.3 Files from Base common folder

- ..\Base_TS_T40D34M20I0R0\include\
- ..\Base_TS_T40D34M20I0R0\header\
- ..\Base_TS_T40D34M20I0R0\src\OsIf_Timer.c
- ..\Base_TS_T40D34M20I0R0\src\OsIf_Timer_System.c

### 3.2.4 Files from Det folder:

- ..\Det_TS_T40D34M20I0R0\include\Det.h
- ..\Det_TS_T40D34M20I0R0\src\Det.c

### 3.2.5 RTE Files:

- Rte_TS_T40D34M20I0R0\include\SchM_Crc.h
- Rte_TS_T40D34M20I0R0\src\SchM_Crc.c

## 3.3 Setting up the plugins

The CRC driver was designed to be configured by using the EB Tresos Studio (version EB tresos Studio 28.0.0 or later.)

#### 3.3.0.0.1 Steps to generate the configuration:

1. Copy the following module folders into the Tresos plugins folder:
   - Base_TS_T40D34M20I0R0
   - Det_TS_T40D34M20I0R0
   - Resource_TS_T40D34M20I0R0
   - Rte_TS_T40D34M20I0R0
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the EB tresos Studio GUI to modify ECU configuration parameters values.
4. Generate the configuration files.

#### 3.3.0.0.2 Location of various files inside the RM module folder:

- VSMD (Vendor Specific Module Definition) file in EB Tresos Studio XDM format:
  - Crc_TS_T40D34M20I0R0\config\Crc.xdm
- VSMD (Vendor Specific Module Definition) file(s) in AUTOSAR compliant EPD format:
  - Crc_TS_T40D34M20I0R0\autosar\Crc_<subderivative_name>.epd
- Code Generation Templates :
  - Source files:
    - ..\Crc_TS_T40D34M20I0R0\generate_PC\src\Crc_Cfg.c
    - ..\Crc_TS_T40D34M20I0R0\generate_PC\src\Crc_Ip_Cfg.c
  - Include file:
    - ..\Crc_TS_T40D34M20I0R0\generate_PC\include\Crc_Cfg.h
    - ..\Crc_TS_T40D34M20I0R0\generate_PC\include\Crc_Ip_Cfg.h
    - ..\Crc_TS_T40D34M20I0R0\generate_PC\include\Crc_Ip_CfgDefines.h

# Chapter 4

# Function calls to module


- Function Calls during Start-up
- Function Calls during Shutdown
- Function Calls during Wake-up


## 4.1 Function Calls during Start-up

Crc_Init()

## 4.2 Function Calls during Shutdown

None.

## 4.3 Function Calls during Wake-up

None.

# Chapter 5

# Module requirements


- Exclusive areas to be defined in BSW scheduler
- Exclusive areas not available on this platform
- Peripheral Hardware Requirements
- ISR to configure within AutosarOS - dependencies
- ISR Macro
- Other AUTOSAR modules - dependencies
- Data Cache Restrictions
- User Mode support
- Multicore support


## 5.1 Exclusive areas to be defined in BSW scheduler

In the current implementation, CRC does not use Schedule Manager (SchM) services.

## 5.2 Exclusive areas not available on this platform

None.

## 5.3 Peripheral Hardware Requirements

None.

## 5.4 ISR to configure within AutosarOS - dependencies

The CRC module does not have an ISR.

## 5.5 ISR Macro

RTD drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions.

### 5.5.1 Without an Operating System

The macro *USING_OS_AUTOSAROS* must not be defined.

#### 5.5.1.1 Using Software Vector Mode

The macro *USE_SW_VECTOR_MODE* must be defined and the ISR macro is defined as:

```
#define ISR(IsrName) void IsrName(void)
```

In this case, the drivers' interrupt handlers are normal C functions and their prologue/epilogue will handle the context save and restore.

#### 5.5.1.2 Using Hardware Vector Mode

The macro *USE_SW_VECTOR_MODE* must not defined and the ISR macro is defined as:

```
#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)
```

In this case, the drivers' interrupt handlers must also handle the context save and restore.

### 5.5.2 With an Operating System

Please refer to your OS documentation for description of the ISR macro.

## 5.6 Other AUTOSAR modules - dependencies

```
- Det: The DET module is used for enabling Development error detection.
       The API function used is Det_ReportError(). The activation / deactivation of Development error
       detection is configurable using the 'McuDevErrorDetect' configuration parameter.

- Rte: The RTE module is needed for implementing data consistency
       of exclusive areas that are used by MCU module. The module is the realization (for a particular ECU)
       of the interfaces of the AUTOSAR Virtual Function Bus (VFB) and thus provides the infrastructure services
       for communication between Application Software Components as well as facilitating access to basic software
       components including the OS

- Base: The BASE module contains the common files/definitions needed by the MCAL.
       This means that it is a dependency for all other MCAL modules.

- Resource: Resource module is used to select microcontroller's derivatives

- MCL: MCL module is used to copy the data from the buffer to the Data Register using DMA.
```

## 5.7 Data Cache Restrictions

In the DMA transfer mode, DMA transfers may issue cache coherency problems. To avoid possible coherency issues when D-CACHE is enabled, the user shall ensure that the buffers used as TCD source and destination are allocated in the NON-CACHEABLE area (by means of CRC_Memmap).

## 5.8 User Mode support

- User Mode configuration in the module
- User Mode configuration in AutosarOS

### 5.8.1 User Mode configuration in the module

The CRC can be run in user mode if the following steps are performed:

- Enable **CrcEnableUserModeSupport** from the configuration
- Call the following functions as trusted functions:

| Function syntax | Description | Available via |
|---|---|---|
| void Crc_Ip_SetUserAccess↩ Allowed(void) | This function configure Set User Access Allowed | Crc_Ip_Trusted↩ Functions.h |

### 5.8.2 User Mode configuration in AutosarOS

When User mode is enabled, the driver may has the functions that need to be called as trusted functions in AutosarOS context. Those functions are already defined in driver and declared in the header `<IpName>_Ip↩ _TrustedFunctions.h`. This header also included all headers files that contains all types definition used by parameters or return types of those functions. Refer the chapter User Mode configuration in the module for more detail about those functions and the name of header files they are declared inside. Those functions will be called indirectly with the naming convention below in order to AutosarOS can call them as trusted functions.

`Call_<Function_Name>_TRUSTED(parameter1,parameter2,...)`

That is the result of macro expansion `OsIf_Trusted_Call` in driver code:

`#define` OsIf_Trusted_Call[1-6params](name,param1,...,param6) Call_##name##_TRUSTED(param1,...,param6)

So, the following steps need to be done in AutosarOS:

- Ensure `MCAL_ENABLE_USER_MODE_SUPPORT` macro is defined in the build system or somewhere global.
- Define and declare all functions that need to call as trusted functions follow the naming convention above in Integration/User code. They need to visible in `Os.h` for the driver to call them. They will do the marshalling of the parameters and call `CallTrustedFunction()` in OS specific manner.
- `CallTrustedFunction()` will switch to privileged mode and call `TRUSTED_<Function_Name>()`.

- `TRUSTED_<Function_Name>()` function is also defined and declared in Integration/User code. It will unmarshalling of the parameters to call `<Function_Name>`() of driver. The `<Function_Name>`() functions are already defined in driver and declared in `<IpName>_Ip_TrustedFunctions.h`. This header should be included in OS for OS call and indexing these functions.

See the sequence chart below for an example calling `Linflexd_Uart_Ip_Init_Privileged()` as a trusted function.

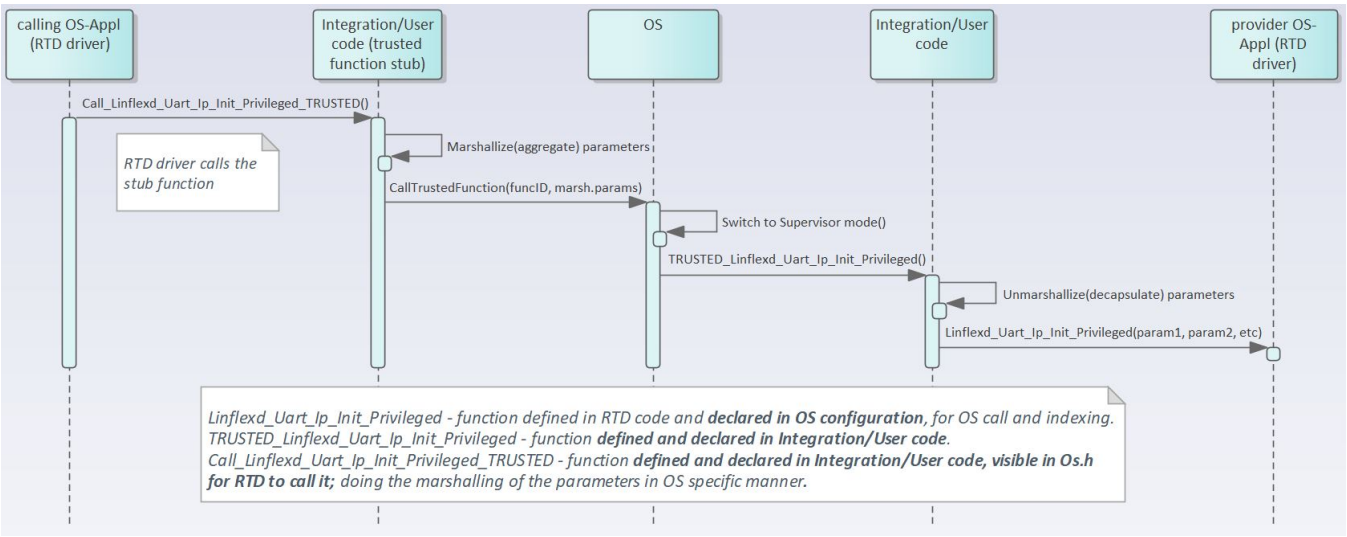



**Figure 5.1 Example sequence chart for calling `Linflexd_Uart_Ip_Init_Privileged` as trusted function**

## 5.9 Multicore support

The CRC implements the "Autosar 4.4 MCAL Multicore Distribution" according to type II, in which the mappable element is set to HW Unit. For additional details, please refer to AUTOSAR_EXP_BSWDistributionGuide. The user can enable the "CRC Multicore Enable" checkbox. Then the user can configure each channel to be controlled by only one core.

# Chapter 6

# Main API Requirements


- Main function calls within BSW scheduler
- API Requirements
- Calls to Notification Functions, Callbacks, Callouts


## 6.1 Main function calls within BSW scheduler

None.

## 6.2 API Requirements

None.

## 6.3 Calls to Notification Functions, Callbacks, Callouts

None.

# Chapter 7

# Memory allocation

- Sections to be defined in CRC_MemMap.h
- Linker command file

## 7.1 Sections to be defined in CRC_MemMap.h

| Section name | Type of section | Description |
|---|---|---|
| CRC_START_SEC_CODE | Code | Start of memory Section for Code |
| CRC_STOP_SEC_CODE | Code | End of memory Section for Code |
| CRC_START_SEC_VAR_CLEARED_↩ _UNSPECIFIED | Variables | Used for variables, structures, arrays when the SIZE (alignment) does not fit the criteria of 8,16 or 32 bit. These variables are cleared to zero by start-up code. |
| CRC_STOP_SEC_VAR_CLEARED_↩ UNSPECIFIED | Variables | End of above section. |
| CRC_START_SEC_VAR_INIT_UNS↩ PECIFIED | Variables | Used for variables, structures, arrays when the SIZE (alignment) does not fit the criteria of 8,16 or 32 bit. These variables are initialized with values after every reset. |
| CRC_STOP_SEC_VAR_INIT_UNSP↩ ECIFIED | Variables | End of above section. |
| CRC_START_SEC_CONST_8 | Constant Data | Used for constants to be aligned to 8 bit. For instance used for constants of size 8 bit or used for composite data types: arrays, structs containing elements of maximum 8 bits. |
| CRC_STOP_SEC_CONST_8 | Constant Data | End of above section. |
| CRC_START_SEC_CONST_16 | Constant Data | Used for constants to be aligned to 16 bit. For instance used for constants of size 16 bit or used for composite data types: arrays, structs containing elements of maximum 16 bits. |
| CRC_STOP_SEC_CONST_16 | Constant Data | End of above section. |

| Section name | Type of section | Description |
|---|---|---|
| CRC_START_SEC_CONST_32 | Constant Data | Used for constants to be aligned to 32 bit. For instance used for constants of size 32 bit or used for composite data types: arrays, structs containing elements of maximum 32 bits. |
| CRC_STOP_SEC_CONST_32 | Constant Data | End of above section. |
| CRC_START_SEC_CONST_UNSPE↩ CIFIED | Constant Data | Used for constants that does not fit the criteria of 8,16 or 32 bit. |
| CRC_STOP_SEC_CONST_UNSPECI↩ FIED | Constant Data | End of above section. |
| CRC_START_SEC_CONFIG_DATA↩ _UNSPECIFIED | Configuration Data | Start of Memory Section for Config Data |
| CRC_STOP_SEC_CONFIG_DATA_↩ UNSPECIFIED | Configuration Data | End of above section. |

## 7.2 Linker command file

Memory shall be allocated for every section defined in the driver's "<Module>"_MemMap.h.

# Chapter 8

# Integration Steps

This section gives a brief overview of the steps needed for integrating this module:

1. Generate the required module configuration(s). For more details refer to section Files Required for Compilation
2. Allocate the proper memory sections in the driver's memory map header file ("<Module>"_MemMap.h) and linker command file. For more details refer to section Sections to be defined in <Module>_MemMap.h
3. Compile & build the module with all the dependent modules. For more details refer to section Building the Driver

# Chapter 9

# External assumptions for driver

The section presents requirements that must be complied with when integrating the CRC driver into the application.

| External Assumption Req ID | External Assumption Text |
|---|---|
| EA_RTD_00082 | When caches are enabled and data buffers are allocated in cacheable memory regions the buffers involved in DMA transfer shall be aligned with both start and end to cache line size. Note: **Rationale**: This ensures that no other buffers/variables compete for the same cache lines. |
| EA_RTD_00092 | The integrator shall allocate a single EcucPartition per core or the partition in which the Crc is allocated shall be exclusively mapped to a core. Note↩ : Internally, the Crc will use the Core ID returned by GetCoreID API to reference the appropriate global data and configuration elements, that is why a core should reference only one configured partition. |
| EA_RTD_00093 | The application shall define EcucCoreIDs in a compact/consecutive order, starting from zero. |
| EA_RTD_00095 | The application shall call Crc_Init() on a single, designated core, using a single configuration pointer. |
| EA_RTD_00096 | The application shall pass the correct initialization pointer, specific to the partition in which the driver is to be used. |
| EA_RTD_00106 | Standalone IP configuration and HL configuration of the same driver shall be done in the same project |
| EA_RTD_00107 | The integrator shall use the IP interface only for hardware resources that were configured for standalone IP usage. Note: The integrator shall not directly use the IP interface for hardware resources that were allocated to be used in HL context. |
| EA_RTD_00108 | The integrator shall use the IP interface to a build a CDD, therefore the BSWMD will not contain reference to the IP interface |

***How to Reach Us:***

**Home Page:**
nxp.com

**Web Support:**
nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, AltiVec, C–5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C–Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2022 NXP B.V.